

NEURAL PLATFORM

NS Backend web API integration

V.26.1



Description:

Web API endpoints description for external system integration.

+34 93 820 56 94
info@neurallabs.net

Version

Version	Description	Developer	Date
1.0	Create document	Kiko	11/12/2024
1.1	Add Section methods	Kiko	06/03/2025
1.2	Add ExclusiveLane in GetPenaltyDetail and PenaltyReasons in GetPenalties	Raúl P.	10/04/2025
1.3	Set the required parameters in the method section and assign default values	Raúl P.	14/04/2025
1.4	Update video endpoints using a generic image controller	Raúl P	06/11/2025
1.5	26.1 - Remove obsolete methods	Kiko	12/02/2026
1.6	26.1 – Add Events and images methods	Kiko	23/02/2026
1.7	26.1 – Add Get penalties from ID	Kiko	15/05/2026

Index

1.	Objective.....	1
2.	Login endpoint.....	2
3.	Penalties endpoints	5
3.1.	Get Penalties.....	5
3.2.	Get Penalties from ID	9
3.3.	Get penalty detail	13
4.	Image & Video	24
4.1.	Get ANPR image	24
4.2.	Get Evidence images by type	28
4.3.	Get Video	32
4.4.	Download Video	36
5.	Sections.....	38
5.1.	Get sections calculation.....	38
6.	Events	44
6.1.	Get Events.....	44
6.2.	Get event by identifier.....	52

1. Objective

The objective of this document is to provide customers with the ability to integrate external systems with the Neural Platform backend for information exchange. In the following chapters will explain the web API endpoints in detail.

2. Login endpoint

All endpoint in the document needs a token before get information from web API. The token must include in the header of the endpoint call. To obtain the access token the client must make the following call.

- **Verb:** POST
- **Path:** /nlapi/api/v2/Auth/login
- **Params:** in JSON format.

```
{  
  "username": "integrationUser",  
  "password": "1234",  
  "keepMeLoggedIn": false,  
  "verificationCode": "",  
  "language": "en"  
}
```

- **Response:**

```
{  
  "data": {  
    "token": "eyJhbGciOiJIUzI1NiIs...",  
    "refreshToken": "eyJhbGciOiJIUzI1NiIs...",  
    "isAdmin": true,  
    "keepMeLoggedIn": false,  
    "twoFactorRequired": false,  
    "setPasswordRequired": false,  
    "userInfo": {  
      "username": "integrationUser"  
    }  
  },  
  "isSuccess": true,  
  "type": null,  
  "errors": []  
}
```

Each endpoint call must have an “Authorization” : “Bearer ” + token in the header.

▪ **Response errors:**

- 400 Bad Request (validation error).

Returned when username or password is missing.

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "Validation",
  "errors": [
    {
      "code": "Login.UsernameRequired",
      "detail": "Username is required."
    },
    {
      "code": "Login.PasswordRequired",
      "detail": "Password is required."
    }
  ]
}
```

- 401 Unauthorized.

Returned when credentials are invalid or user cannot authenticate.

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "Unauthorized",
  "errors": [
    {
```

Neural Platform – NS Backend web API integration

```
"code": "Login.InvalidCredentials",
"detail": "Invalid username or password."
}
]
}
```

- 422 Unprocessable Entity.

Returned for business authentication cases such as invalid two-factor code.

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "Unprocessable",
  "errors": [
    {
      "code": "Login.InvalidCode",
      "detail": "Invalid Code."
    }
  ]
}
```

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

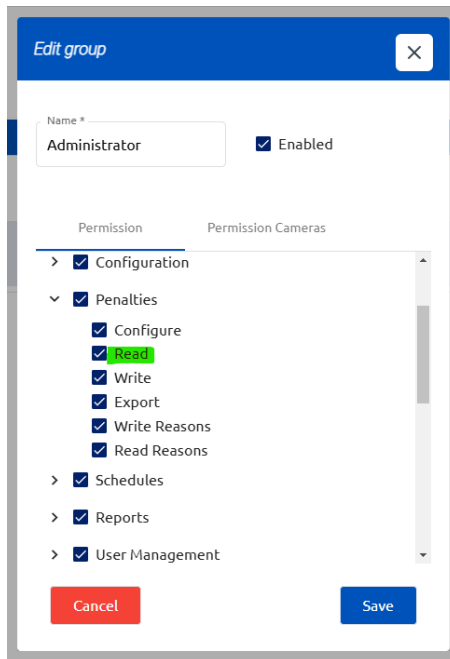
3. Penalties endpoints

The following chapters detail the endpoints related to penalties.

3.1. Get Penalties

The endpoint to get a penalties list return a penalty list with information filtered by dates from / to. The next table has a parameters detail list. Parameters marked with “*” are mandatory in the request.

- **Verb:** POST
- **Path:** /nlapi/api/v2/Penalties/get-penalties
- **Permissions:** login user needs permission with name command “readpenalties”



Parameters:

Param name	Type	Default values	Format or allowed values	Description
* dateFrom	DateTime	2026-02-24T12:11:46	2026-02-24T12:11:46 or 2026-02-24 12:11:46	Date from filter
* dateTo	DateTime	2026-02-24T23:59:59	2026-02-24T23:59:59 or 2026-02-24 23:59:59	Date to filter
typesId	List<int?>	[]	[15, 2, 4]	Unique penalty type identifier list
cameras	Dictionary<string,string>	{}	{ "1": "1", "4": "1" }	Camera identifier list. Key = cameraId, Value = computerId
numberPlate	string		Exact plate or regex with ?, * and operators AND, OR	Number plate filter
statusId	List<int?>	[]	[2, 3, 4]	Unique status identifier list
minutesFrom	int?			Only for Restricted area penalty. Minutes in a restricted area from
minutesTo	int?			Only for Restricted area penalty. Minutes in a restricted area to
reasonsId	Dictionary<int,List<int>>?	{}	{ "5": [4, 1, 2, 13, 6] }	Key = penaltyId, value = list of associated reason IDs

Response detail:

Result envelope:

- data: response payload
- isSuccess: boolean.
- type: string or null.
- errors: array of error items.

data:

- penaltyId: (PK) (integer) penalty identifier.
- typeId: (integer) penalty type identifier.
- typeDescription: (string) penalty type internal code name.
- numberPlate: (string) number plate.
- numberPlateExport: (string) plate number prepared for export.
- globalConfidence: (double) reliability percentage.
- date: (datetime) penalty date time.

- reasonId: (integer) penalty reason identifier.
 - reason: (string) penalty reason internal code name.
 - customReason: (string) used only when user adds a custom reason.
 - statusId: (integer) penalty status identifier.
 - statusDescription: (string) penalty status internal code name.
 - minutesPenalty: (int) only for restricted area. Minutes affected in restricted area.
- Default null.
- statusDate: (datetime) last status update date time.
 - diffMinutes: (int) only for restricted area. Minutes in restricted area. Default null.
 - configurationId: (int) not used.
 - camerasNames: (string) camera name.
 - observations: (string) observations text.

- **data.totalRows:** (int) total result count.

▪ **Response example:**

```
{
  "data": {
    "items": [
      {
        "penaltyId": 18,
        "typeId": 2,
        "typeDescription": "FOTOROJO",
        "numberPlate": "9280HYW",
        "numberPlateExport": "9280HYW",
        "globalConfidence": 99.9000015258789,
        "date": "2024-12-10T10:48:45.137",
        "reasonId": 7,
        "reason": "SKIP TRAFFIC LIGHT",
        "customReason": "",
        "statusId": 1,
        "statusDescription": "PROPUESTA",
        "minutesPenalty": null,
        "statusDate": "2024-12-10T10:51:11.343",
        "diffMinutes": null,
        "configurationId": 1,
        "camerasNames": "AjuntaCam_1",
        "observations": null
      }
    ]
  }
}
```

```
}  
],  
  "totalRows": 2  
},  
  "isSuccess": true,  
  "type": null,  
  "errors": []  
}
```

Response errors:

- 400 Bad Request (validation error).

Returned when request validation fails (for example: dateFrom is required, dateTo is required, or dateFrom >= dateTo).

Response example:

```
{  
  "data": null,  
  "isSuccess": false,  
  "type": "Validation",  
  "errors": [  
    {  
      "code": "ExportPenalties.DateFromRequired",  
      "detail": "DateFrom is required."  
    },  
    {  
      "code": "ExportPenalties.DateToRequired",  
      "detail": "DateTo is required."  
    },  
    {  
      "code": "ExportPenalties.DateRangeInvalid",  
      "detail": "DateFrom must be less than DateTo."  
    }  
  ]  
}
```

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid/expired token).

Response body may be empty.

- 403 Forbidden.

Returned when the token is valid but does not contain required permission ("readpenalties").

Response body may be empty.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

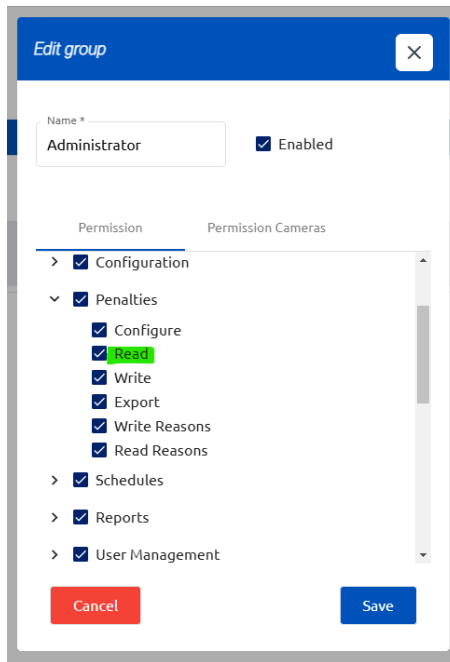
```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

3.2. Get Penalties from ID

The endpoint to get penalties from ID, Returns a list of up to one hundred penalties based on the provided identifier. The provided identifier is also included in the response.

- **Verb:** GET
- **Path:** /nlapi/api/v2/Penalties/get-penalties-from-id
- **Permissions:** login user needs permission with name command “readpenalties”

Neural Platform – NS Backend web API integration



Parameters:

Param name	Type	Default values	Format or allowed values	Description
* fromId	int			Id number

Response detail:

• Result envelope:

- data: response payload
- isSuccess: boolean.
- type: string or null.
- errors: array of error items.

• data:

- penaltyId: (PK) (integer) penalty identifier.
- typeId: (integer) penalty type identifier.
- typeDescription: (string) penalty type internal code name.
- numberPlate: (string) number plate.
- numberPlateExport: (string) plate number prepared for export.
- globalConfidence: (double) reliability percentage.

- date: (datetime) penalty date time.
- reasonId: (integer) penalty reason identifier.
- reason: (string) penalty reason internal code name.
- customReason: (string) used only when user adds a custom reason.
- statusId: (integer) penalty status identifier.
- statusDescription: (string) penalty status internal code name.
- minutesPenalty: (int) only for restricted area. Minutes affected in restricted area.

Default null.

- statusDate: (datetime) last status update date time.
- diffMinutes: (int) only for restricted area. Minutes in restricted area. Default null.
- configurationId: (int) not used.
- camerasNames: (string) camera name.
- observations: (string) observations text.

- **data.totalRows:** (int) total result count.

▪ **Response example:**

```
{
  "data": {
    "items": [
      {
        "penaltyId": 18,
        "typeId": 2,
        "typeDescription": "FOTOROJO",
        "numberPlate": "9280HYW",
        "numberPlateExport": "9280HYW",
        "globalConfidence": 99.9000015258789,
        "date": "2024-12-10T10:48:45.137",
        "reasonId": 7,
        "reason": "SKIP TRAFFIC LIGHT",
        "customReason": "",
        "statusId": 1,
        "statusDescription": "PROPUESTA",
        "minutesPenalty": null,
        "statusDate": "2024-12-10T10:51:11.343",
        "diffMinutes": null,
        "configurationId": 1,
        "camerasNames": "AjuntaCam_1",
        "observations": null
      }
    ]
  }
}
```

```
}  
],  
  "totalRows": 2  
},  
  "isSuccess": true,  
  "type": null,  
  "errors": []  
}
```

Response errors:

- 400 Bad Request (validation error).

Returned when request validation fails (for example: dateFrom is required, dateTo is required, or dateFrom >= dateTo).

Response example:

```
{  
  "data": null,  
  "isSuccess": false,  
  "type": "Validation",  
  "errors": [  
    {  
      "code": " ExportPenaltiesFromIdErrorCodes.FromIdInvalid",  
      "detail": "FromId must be greater than zero."  
    }  
  ]  
}
```

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid/expired token).

Response body may be empty.

- 403 Forbidden.

Returned when the token is valid but does not contain required permission ("readpenalties").

Response body may be empty.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

3.3. Get penalty detail

The endpoint to get the detail of a penalty has the following configuration.

- **Verb:** GET
- **Path:** /nlapi/api/v2/Penalties/get-penalty-by-id
- **Permissions:** login user needs permission with name command "readpenalties"
- **Parameters:**

Param name	Type	Default values	Format or allowed values	Description
* penaltyId	Int			Penalty identifier.
addImagePath	boolean	false	false / true	false = return binary image. true = return image path.
- **Response detail:**
- **Result envelope:**

- data: response payload.
- isSuccess: boolean.
- type: string or null.
- errors: array of error items.

▪ **Data:**

- penaltyId: (int) penalty identifier.
- typeId: (byte) penalty type identifier.
- typeDescription: (string) penalty type internal code name.
- numberPlate: (string) number plate.
- numberPlateExport: (string) plate number prepared for export.
- globalConfidence: (double) reliability percentage.
- date: (datetime) penalty date time.
- reasonId: (int) penalty reason identifier.
- reason: (string) penalty reason internal code name.
- customReason: (string) used only when user adds a custom reason.
- statusId: (byte) penalty status identifier.
- statusDescription: (string) penalty status internal code name.
- minutesPenalty: (int) only for restricted area. Minutes affected. Default null.
- statusDate: (datetime) last status update date time.

- diffMinutes: (int) only for restricted area. Minutes in restricted area. Default null.
- configurationId: (int) not used.
- camerasNames: (string) camera name.
- observations: (string) observations text.
- totalRows: (int) total result count.
- data.events: array of events/incidences related to the penalty.
 - eventId: (int) event identifier.
 - eventDate: (datetime) date time when the camera detected the event.
 - numberPlate: (string) plate number.
 - cameraId: (int) camera identifier.
 - cameraName: (string) camera name.
 - computerId: (int) computer identifier.
 - cutImagePath: (string) cut image path (when addImagePath = true).
 - cutImage: (string) cut image base64 (when addImagePath = false).
 - anprImagePath: (string) ANPR image path (when addImagePath = true).
 - anprImage: (string) ANPR image base64 (when addImagePath = false).
 - speed: (decimal) speed detected.
 - speedConfidence: (double) speed reliability percentage.

- redLightState: (byte) traffic light state.
- trafficLightStatus: (string) traffic light status description.
- vehicleMake: (string) vehicle brand.
- vehicleMakeConfidence: (double) brand reliability percentage.
- direction: (int) lane direction.
- vehicleClassDescription: (string) classification description.
- locationId: (int) location identifier.
- evidencelImages: array of evidence images for the event.
 - imagePath: (string) evidence image path.
 - imageBytes: (string) evidence image base64.
- data.instantSpeed: instant speed penalty values. Null if not applicable.
 - speed: (decimal) vehicle speed.
 - speedConfidence: (double) speed reliability.
 - allowedSpeed: (double) allowed speed.
- data.restrictedArea: restricted area penalty values. Null if not applicable.
 - entryDate: (datetime) vehicle entry date time.
 - entryCameraName: (string) entry camera name.
 - exitDate: (datetime) vehicle exit date time.
 - exitCameraName: (string) exit camera name.

- elapsedTimeInMinutes: (double) difference between entry and exit.
- data.illegalStop: illegal stop penalty values. Null if not applicable.
 - cameraName: (string) camera name.
 - detectionDate: (datetime) detection date time.
- data.illegalTurn: illegal turn penalty values. Null if not applicable.
 - entryCameraName: (string) entry camera name.
 - entryIncidenceDate: (datetime) entry incidence date time.
 - exitCameraName: (string) exit camera name.
 - exitIncidenceDate: (datetime) exit incidence date time.
 - time: (string) elapsed time formatted. E.g. '5h', '5m' or '5s'.
- data.wrongWay: wrong way penalty values. Null if not applicable.
 - cameraName: (string) camera name.
 - detectionDate: (datetime) detection date time.
 - detectedDirection: (string) detected direction. Values: Coming, Going or Undetected.
 - configuredDirection: (string) configured direction. Values: Coming, Going or Undetected.
- data.section: section penalty values. Null if not applicable.
 - entryIncidenceDate: (datetime) entry incidence date time.
 - entryCameraName: (string) entry camera name.

- exitDate: (datetime) exit date time.
- exitCameraName: (string) exit camera name.
- detectedSpeed: (double) detected speed.
- data.redLight: red-light penalty values. Null if not applicable.
 - detectionDate: (datetime) detection date time.
 - cameraName: (string) camera name.
 - trafficLightStatus: (string) traffic light status.
 - vehicleMake: (string) vehicle brand.
 - vehicleMakeConfidence: (double) brand reliability.
 - redLightState: (string) red-light state. Internal code name.
- data.trafficJam: traffic jam penalty values. Null if not applicable.
 - cameraName: (string) camera name.
 - detectionDate: (datetime) detection date time.
- data.abandonedObject: abandoned object penalty values. Null if not applicable.
 - cameraName: (string) camera name.
 - detectionDate: (datetime) detection date time.
- data.averageSpeedAlteration: average speed alteration penalty values. Null if not applicable.
 - cameraName: (string) camera name.

- `detectionDate`: (datetime) detection date time.
- `data.clearance`: clearance penalty values. Null if not applicable.
 - `isClearance`: (boolean).
- `data.exclusiveLane`: exclusive lane penalty values. Null if not applicable.
 - `classification`: (string) detected vehicle classification.
- `data.restrictedPlate`: restricted plate penalty values. Null if not applicable.
 - `dateFrom`: (datetime) restriction start date.
 - `dateTo`: (datetime) restriction end date.
 - `regex`: (string) restriction regular expression.
- **Response example:**

This example is for a red-light penalty detail.

```
{
  "data": {
    "penalty": {
      "penaltyId": 18,
      "typeId": 2,
      "typeDescription": "FOTOROJO",
      "numberPlate": "9280HYW",
      "numberPlateExport": "9280HYW",
      "globalConfidence": 99.9000015258789,
      "date": "2024-12-10T10:48:45.137",
      "reasonId": 7,
      "reason": "SKIP TRAFFIC LIGHT",
      "customReason": "",
      "statusId": 1,

```

Neural Platform – NS Backend web API integration

```
"statusDescription": "PROPUESTA",
"minutesPenalty": null,
"statusDate": "2024-12-10T10:51:11.343",
"diffMinutes": null,
"configurationId": 1,
"camerasNames": "AjuntaCam_1",
"observations": null,
"totalRows": 1
},
"events": [
{
  "eventId": 144061,
  "eventDate": "2024-12-10T10:48:45.137",
  "numberPlate": "9280HYW",
  "cameraId": 1,
  "cameraName": "AjuntaCam_1",
  "computerId": 1,
  "cutImagePath": "",
  "cutImage": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQEAYABg .....",
  "anprImagePath": "",
  "anprImage": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQEAYABg .....",
  "speed": -1.00,
  "speedConfidence": -1,
  "redLightState": 3,
  "trafficLightStatus": "RED",
  "vehicleMake": "FIAT",
  "vehicleMakeConfidence": 70.61456298828125,
  "direction": 1,
  "vehicleClassDescription": "VAN",
  "locationId": 1,
  "evidenceImages": [
    {
      "imagePath": "",
      "imageBytes": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQEAYABg ....."
    }
  ]
}
```

```
]
}
],
"instantSpeed": null,
"restrictedArea": null,
"illegalStop": null,
"illegalTurn": null,
"wrongWay": null,
"section": null,
"redLight": {
  "detectionDate": "2024-12-10T10:48:45.137",
  "cameraName": "AjuntaCam_1",
  "trafficLightStatus": null,
  "vehicleMake": "FIAT",
  "vehicleMakeConfidence": 70.61456298828125,
  "redLightState": "RED"
},
"trafficJam": null,
"abandonedObject": null,
"averageSpeedAlteration": null,
"clearance": null,
"exclusiveLane": null,
"restrictedPlate": null
},
"isSuccess": true,
"type": null,
"errors": []
}
```

If customer wants to get the incidence video for the penalty a second call must be done to the endpoint "Video".

- **Response errors:**

- 400 Bad Request (validation error).

Returned when penaltyId is invalid (≤ 0).

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "Validation",
  "errors": [
    {
      "code": "GetPenaltyById.PenaltyIdInvalid",
      "detail": "PenaltyId must be greater than 0."
    }
  ]
}
```

- 404 Not Found.

Returned when the provided penaltyId does not exist.

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "NotFound",
  "errors": [
    {
      "code": "GetPenaltyById.PenaltyNotFound",
      "detail": "penaltyId not found"
    }
  ]
}
```

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid/expired token).

Response body may be empty.

- 403 Forbidden.

Returned when the token is valid but does not contain required permission ("readpenalties").

Response body may be empty.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

4. Image & Video

4.1. Get ANPR image

The endpoint to get an ANPR image has the following configuration.

- **Verb:** GET
- **Path:** /nlapi/api/v2/ImageVideo/get-anpr-images
- **Permissions:** get anpr images needs permission with name command "readpenalties", "ver_events", "view_sections", "convoy_analysis", "consultar_alarmas".
- **Parameters:**

Param name	Type	Default values	Format or allowed values	Description
* eventId	int			Event identifier
* computerId	int			Computer identifier
includeCutImage	Boolean	true	true / false	If true returns image and cut image. If false returns only image

- **Response detail:**
 - **Result envelope:**
 - data: response payload
 - isSuccess: boolean.
 - type: string or null.
 - errors: array of error items.

Data:

eventId: (int) event identifier.

computerId: (int) computer identifier.

imageVideoid: (int) image identifier.

cameraId: (int) camera identifier.

cameraName: (string) camera name.

imageBytes: (string) base64 encoded image.

imageType: (string) image type. 'Lpr' for ANPR image, 'Cut' for cut image (only when includeCutImage=true).

imagePath: (string) image file path on the server.

imageBoundingBoxes: (array) bounding box coordinates.

top: (int?) top coordinate.

bottom: (int?) bottom coordinate.

left: (int?) left coordinate.

right: (int?) right coordinate.

boundingBoxType: (string) type of bounding box. For 'Lpr' images possible values include "classification", "cut", and "radar". For 'Cut' images, imageBoundingBoxes is an empty array.

▪ Response example:

```
{  
  "data": [  
    ...  
  ]  
}
```

```
{
  "eventId": 174132,
  "computerId": 1,
  "imageVideoId": 184801,
  "cameraId": 4,
  "cameraName": "Ajunta-Video-1",
  "imageBytes": "data:image/jpeg;base64,/9j/4AAQSkZJR...",
  "imageType": "Lpr",
  "imagePath": "c:\\tmp\\imgs\\RH415.jpg",
  "imageBoundingBoxes": [
    {
      "top": 427,
      "bottom": 451,
      "left": 910,
      "right": 1030,
      "boundingBoxType": "plate"
    }
  ]
},
{
  "isSuccess": true,
  "type": null,
  "errors": []
}
```

- **Response errors:**

- 400 Bad Request (validation error).

Returned when one or more parameters are invalid (for example: eventId <= 0 or computerId <= 0).

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "Validation",
  "errors": [
    {
      "code": "GetAnprImages.EventIdInvalid",
      "detail": "EventId must be greater than 0."
    },
    {
      "code": "GetAnprImages.ComputerIdInvalid",
      "detail": "ComputerId must be greater than 0."
    }
  ]
}
```

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid/expired token).

Response body may be empty.

- 403 Forbidden.

Returned when the token is valid but does not contain required permission.

Response body may be empty.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
  }
}
```

```

    "data": "",
    "exception": "Exception"
  }
}

```

4.2. Get Evidence images by type

The endpoint to get evidence images by type for an event has the following configuration.

- **Verb:** GET
- **Path:** /nlapi/api/v2/ImageVideo/get-evidence-images-by-type
- **Permissions:** get evidence images needs permission with name command "readpenalties", "ver_events", "view_sections", "convoy_analysis", "consultar_alarmas".
- **Parameters:**

Param name	Type	Default values	Format or allowed values	Description
* eventId	int			Event identifier
* computerId	int			Computer identifier
* type	string		CAMERA, CAMERA_NLO, RECORDER, RECORDER_NLO	Type of the evidence

- **Explanation of types:**

CAMERA: Returns the most important image of the image sequence when the source data comes from Neural Server.

CAMERA_NLO: Returns the most important image of the image sequence when the source data comes from Orchestr[AI]tor.

RECORDER: Returns the image instances used to generate the video.

RECORDER_NLO: Returns the image instances used to generate the video when the source data comes from Orchestr[AI]tor.

▪ **Response detail:**

- **Result envelope:**

data: response payload

isSuccess: boolean.

type: string or null.

errors: array of error items.

Data:

eventId: (int) event identifier.

computerId: (int) computer identifier.

imageVideoid: (int) image identifier.

cameraId: (int) camera identifier.

cameraName: (string) camera name.

imageBytes: (string) base64 encoded image.

imageType: (string) image type. 'CAMERA', 'RECORDER', 'CAMERA_NLO', 'RECORDER_NLO'.

imagePath: (string) image file path on the server.

imageBoundingBoxes: (array) bounding box coordinates. Null when no bounding boxes are available.

▪ **Response example:**

```
{
  "data": [
    {
      "eventId": 479,
      "computerId": 1,
      "imageVideoId": 1308,
      "cameraId": 2,
      "cameraName": "EvCam",
      "imageBytes": "data:image/jpeg;base64,/9j/4AAQSkZIRgABAQAAQABAAD...",
      "imageType": "CAMERA",
      "imagePath": "c:\\tmp\\penalties\\20260223\\110\\EV.jpg",
      "imageBoundingBoxes": []
    }
  ],
  "isSuccess": true,
  "type": null,
  "errors": []
}
```

▪ **Response error:**

- 400 Bad Request (validation error).

Returned when one or more parameters are invalid (for example: eventId <= 0, computerId <= 0, type is empty, or type is not supported).

Response example:

```
{
  "data": null,
  "isSuccess": false,
```

```
"type": "Validation",
"errors": [
  {
    "code": "GetEvidenceImagesByType.EventIdInvalid",
    "detail": "EventId must be greater than 0."
  },
  {
    "code": "GetEvidenceImagesByType.ComputerIdInvalid",
    "detail": "ComputerId must be greater than 0."
  },
  {
    "code": "GetEvidenceImagesByType.TypeInvalid",
    "detail": "Type is required."
  },
  {
    "code": "GetEvidenceImagesByType.TypeNotSupported",
    "detail": "Type must be one of the allowed values."
  }
]
```

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid/expired token).

Response body may be empty.

- 403 Forbidden.

Returned when the token is valid but does not contain required permission.

Response body may be empty.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

4.3. Get Video

The endpoint to get the path of an evidence video has the following configuration.

- **Verb:** GET
- **Path:** /nlapi/api/v2/ImageVideo/get-evidence-videos
- **Permissions:** get evidence videos needs permission with name command "readpenalties", "ver_events", "view_sections", "convoy_analysis", "consultar_alarmas".
- **Parameters:**

Param name	Type	Default values	Format or allowed values	Description
* eventId	int			Event identifier
* computerId	int			Computer identifier
* type	string		RECORDERAVI, RECORDER, RECORDER_NLO	Type of the evidence. To obtain the video use "RECORDERAVI"

- **Explanation of types:**

RECORDERAVI: Returns the video object generated by the recording process.

RECORDER: Returns the image instances used to generate the video.

RECORDER_NLO: Returns the image instances used to generate the video when the source data comes from NLO.

▪ **Response detail:**

• **Result envelope:**

- data: response payload
- isSuccess: boolean.
- type: string or null.
- errors: array of error items.

Data:

eventId: (int) event identifier.

computerId: (int) computer identifier.

imageVideoId: (int) image/video identifier. Use this value in Download Video.

cameraId: (int) camera identifier.

cameraName: (string) camera name.

imageBytes: (string) null for RECORDERAVI type. Use Download Video endpoint to get the file.

imageType: (string) video type. 'RECORDERAVI', 'RECORDER', 'RECORDER_NLO'.

imagePath: (string) video file path on the server.

▪ Response example:

```
{
  "data": [
    {
      "eventId": 80685,
      "computerId": 1,
      "imageVideoid": 121729,
      "cameraId": 8,
      "cameraName": "Camera EV CONTRA",
      "imageBytes": "",
      "imageType": "RECORDERAVI",
      "imagePath": "c:\\tmp\\penalties\\20251105\\72744\\80685-1-3632GDX.mp4"
    }
  ],
  "isSuccess": true,
  "type": null,
  "errors": []
}
```

▪ Response errors:

- 400 Bad Request (validation error).

Returned when one or more parameters are invalid (for example: eventId <= 0, computerId <= 0, type is empty, or type is not supported).

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "Validation",
  "errors": [
    {
      "code": "GetEvidenceVideos.EventIdInvalid",
      "detail": "EventId must be greater than 0."
    }
  ]
}
```

Neural Platform – NS Backend web API integration

```
"code": "GetEvidenceVideos.ComputerIdInvalid",
"detail": "ComputerId must be greater than 0."
},
{
  "code": "GetEvidenceVideos.TypeInvalid",
  "detail": "Type is required."
},
{
  "code": "GetEvidenceVideos.TypeNotSupported",
  "detail": "Type must be one of the allowed values."
}
]
}
```

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid/expired token).

Response body may be empty.

- 403 Forbidden.

Returned when the token is valid but does not contain required permission.

Response body may be empty.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

4.4. Download Video

The endpoint to download an evidence video file has the following configuration.

- **Verb:** GET
- **Path:** /nlapi/api/v2/ImageVideo/download-video
- **Permissions:** download video needs permission with name command "readpenalties", "ver_events", "view_sections", "convoy_analysis", "consultar_alarmas".
- **Parameters:**

Param name	Type	Default values	Format or allowed values	Description
* imageVideoid	int			Evidence video identifier obtained in GetEvidenceVideos()

The response is a file stream (binary). The content type is "video/mp4". Use the imageVideoid obtained from Get Video (evidence-videos) with type "RECORDERAVI".

- **Response:**

The response is a binary file stream. Content-Type is "video/mp4".

- **client example:**

```
public async Task<bool> DownloadVideoAsync(int imageVideoid, string savePath)
{
    string url = $"http://localhost:5000/nlapi/api/v2/ImageVideo/download-
video?imageVideoid={imageVideoid}";

    HttpResponseMessage response = await _httpClient.GetAsync(url);
```

```
if (response.IsSuccessStatusCode)
{
    byte[] videoBytes = await response.Content.ReadAsByteArrayAsync();
    await File.WriteAllBytesAsync(savePath, videoBytes);
    return true;
}

return false;
}
```

▪ **Response errors:**

- 400 Bad Request (validation error).
Returned when imageVideoId is invalid (≤ 0).

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "Validation",
  "errors": [
    {
      "code": "DownloadVideo.ImageVideoIdInvalid",
      "detail": "IdImageVideo must be greater than 0."
    }
  ]
}
```

- 204 No Content.

Returned when the file is not found or when imageVideoId does not correspond to a RECORDERAVI video.

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid/expired token).

Response body may be empty.

- 403 Forbidden.

Returned when the token is valid but does not contain required permission.

Response body may be empty.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

5. Sections

The Web API provides several endpoints to retrieve information about Sections. The following chapters explain how to integrate with the endpoints.

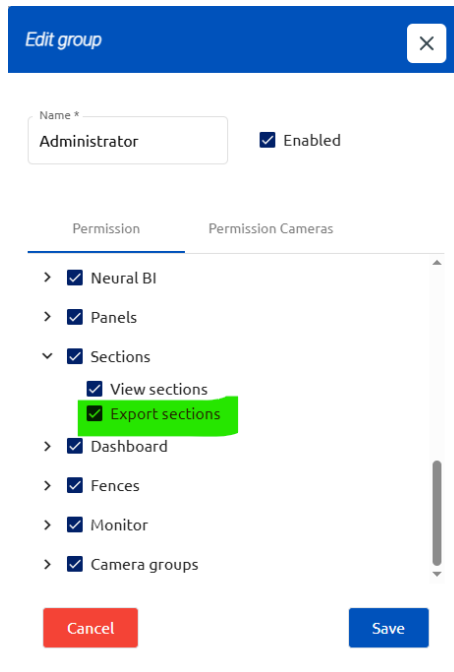
5.1. Get sections calculation

The endpoint to get a section calculation return a section list with information filtered by dates from / to. The next table has a parameters detail list. Parameters marked with “*” are mandatory in the request.

- **Verb:** GET

Neural Platform – NS Backend web API integration

- **Path:** `/nlapi/api/v2/Sections/get-section`
- **Permissions:** login user needs permission with name command “export_sections”.



- **Parameters:**

Param name	Type	Default values	Format or allowed values	Description
* dateFrom	DateTime	2026-02-24T00:00:00	2026-02-24T00:00:00 or 2026-02-24 00:00:00	Date from filter
* dateTo	DateTime	2026-02-24T23:59:59	2026-02-24T23:59:59 or 2026-02-24 23:59:59	Date to filter
numberPlate	string		Exact plate or regex with ?, * and operators AND, OR	Number plate filter
entryLocations	string		&entryLocations=10L1S10L2	Entry locations. Each location: idLocation + 'L' + idComputer. Multiple joined with 'S'. Example: '10L1S10L2'
entryCameras	string		&entryCameras=13C1S14C1	Entry cameras. Each camera: idCamera + 'C' + idComputer. Multiple joined with 'S'. Example: '13C1S14C1'
exitLocations	string		&exitLocations=10L1S10L2	Exit locations. Each location: idLocation + 'L' + idComputer. Multiple joined with 'S'. Example: '10L1S10L2'

Neural Platform – NS Backend web API integration

exitCameras	string		&exitCameras=13C1S14C1	Exit cameras. Each camera: idCamera + 'C' + idComputer. Multiple joined with 'S'. Example: '13C1S14C1'
* durationFrom	TimeSpan	00:00:00	&durationFrom=00:00:00	Duration from filter
* durationTo	TimeSpan	23:59:59	&durationTo=23:59:59	Duration to filter
* avgSpeedFrom	decimal	0	Average speed from value	Average speed from filter
* avgSpeedTo	decimal	10000	Average speed to value	Average speed to filter
vehicleTypeIds	List<int>	[]	&vehicleTypeIds=2&vehicleTypeIds=4	Vehicle type ID list
makes	List<string>	[]	&makes=AUDI&makes=BMW	Make filter list
models	List<string>	[]	&models=AUDI##A1&models=AUDI##A3	Model filter. Format: Make + '##' + Model database key
colors	List<string>	[]	&colors=BLACK&colors=BLUE	Color filter list using database key codes

▪ Response detail:

• Result envelope:

- data: response payload
- isSuccess: boolean.
- type: string or null.
- errors: array of error items.

• data array:

- entryDate: (datetime) entry datetime.
- exitDate: (datetime) exit datetime.
- numberPlate: (string) license plate.
- entryLocationName: (string) entry location description.
- entryCameraName: (string) entry camera description.
- exitLocationName: (string) exit location description.
- exitCameraName: (string) exit camera description.
- averageSpeed: (decimal) average speed.

- duration: (string) elapsed time between entry and exit (format HH:mm:ss, e.g. 00:00:12).
- entryClassificationDescription: (string) entry classification description (database key code).
- entryMake: (string) entry make key code.
- entryModel: (string) entry model key code.
- entryColor: (string) entry color key code.
- exitClassificationDescription: (string) exit classification description (database key code).
- exitMake: (string) exit make key code.
- exitModel: (string) exit model key code.
- exitColor: (string) exit color key code.
- entryCutImage: (string - Base64) entry event image.
- exitCutImage: (string - Base64) exit event image.
- entryEventId: (int) entry event identifier.
- exitEventId: (int) exit event identifier.
- entryComputerId: (int) entry computer identifier.
- exitComputerId: (int) exit computer identifier.
- entryLocationId: (int) entry location identifier.
- exitLocationId: (int) exit location identifier.
- entryCameraId: (int) entry camera identifier.
- exitCameraId: (int) exit camera identifier.
- durationSeconds: (int) elapsed time between entry and exit in seconds.
- entryClassificationId: (int) entry classification identifier.
- exitClassificationId: (int) exit classification identifier.

▪ **Response example:**

```
{  
  "data": [  
    {
```

Neural Platform – NS Backend web API integration

```
"entryDate": "2025-03-07T10:32:19.127",
"exitDate": "2025-03-07T10:32:31.710",
"numberPlate": "2264LKR",
"entryLocationName": "Section IN",
"entryCameraName": "Section Cam IN",
"exitLocationName": "Section OUT",
"exitCameraName": "Section Cam OUT",
"averageSpeed": 1.43,
"duration": "00:00:12",
"entryClassificationDescription": "CAR",
"entryMake": "SUBARU",
"entryModel": "UNKNOWN",
"entryColor": "GREY",
"exitClassificationDescription": "CAR",
"exitMake": "HONDA",
"exitModel": "CR-V",
"exitColor": "WHITE",
"entryCutImage": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAA0",
"exitCutImage": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAA0J",
"entryEventId": 291155,
"exitEventId": 291167,
"entryComputerId": 1,
"exitComputerId": 1,
"entryLocationId": 10,
"exitLocationId": 11,
"entryCameraId": 13,
"exitCameraId": 14,
"durationSeconds": 12,
"entryClassificationId": 2,
"exitClassificationId": 2
}
],
"isSuccess": true,
"type": null,
"errors": []
```

```
}
```

Response errors:**- 400 Bad Request (validation error).**

Returned when validation rules fail (for example: `dateFrom > dateTo`, `avgSpeedFrom > avgSpeedTo`, or `durationFrom > durationTo`).

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "Validation",
  "errors": [
    {
      "code": "GetSectionCalcs.DateRangeInvalid",
      "detail": "DateFrom must be less than or equal to DateTo."
    },
    {
      "code": "GetSectionCalcs.AvgSpeedRangeInvalid",
      "detail": "AvgSpeedFrom must be less than or equal to AvgSpeedTo."
    },
    {
      "code": "GetSectionCalcs.DurationRangeInvalid",
      "detail": "DurationFrom must be less than or equal to DurationTo."
    }
  ]
}
```

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid/expired token).

Response body may be empty.

- 403 Forbidden.

Returned when the token is valid but does not contain required permission ("export_sections").

Response body may be empty.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

6. Events

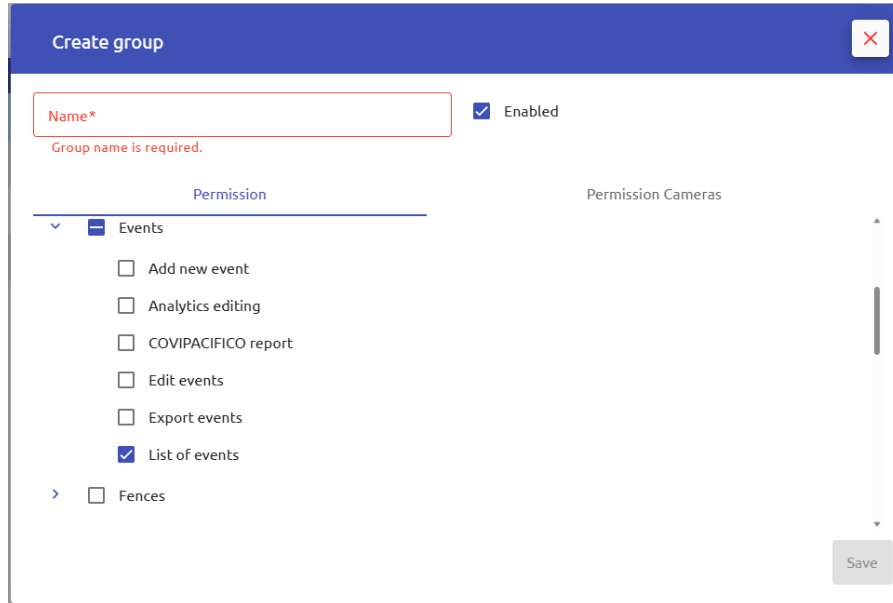
There are several endpoints that can be called to retrieve information about vehicles in the Web API. The following chapters explain how to integrate with the endpoints.

6.1. Get Events

The endpoint to get an events list returns events filtered by dates from / to.. The next table has a parameters detail list. Parameters marked with “*” are mandatory in the request.

Neural Platform – NS Backend web API integration

- **Verb:** *POST*
- **Path:** `/nlapi/api/v2/Events/get-events`
- **Permissions:** login user needs permission with name command “`ver_events`”.



- **Parameters:**

Param name	Type	Default values	Format or allowed values	Description
* dateFrom	string	2026-02-24T12:11:46		Date from filter
* dateTo	string	2026-02-24T12:11:46		Date to filter
numberPlate	string		Exact plate or regex with ?, * and operators AND, OR	Number plate filter
personName	string			Person name filter
camerasId	Array	[]	{ "cameraId": (int), "computerId": (int) }	Camera identifier list
nloEvents	Array	[]	{ "eventName": (string), "cameraId": (int), "computerId": (int) }	NLO event identifier list
computers	Array	[]	[1, 2]	Database ID values, table Computer or null
directions	Array	[]	[1, 2]	Direction ID list or null
countries	Array	[]	[1, 2]	Country ID list or null
locations	Array	[]	{ "locationId": (int), "computerId": (int) }	Location identifier list

Neural Platform – NS Backend web API integration

colors	Array	[]	["BLACK", "BLUE"]	Database ID values, table CNS_COLOR or null
makes	Array	[]	["ACCURA", "AUDI"]	Database description_make, table CNS_MAKES or null
models	Array	[]	{ "makeId": (string), "modelId": (string) }	Make and model identifier
engines	Array	[]	["DIESEL", "GASOLINE"]	Engine type list or null
lightStates	Array	[]	[1, 2]	Light state ID list or null
classifications	Array	[]	[1, 2]	Database ID values, table Type_Vehicle or null
lanes	Array	[]	{ "laneId": (int), "cameraId": (int), "computerId": (int) }	Lane identifier list
includeCutImage	Boolean	true	true / false	True adds cut image as base64 in response

▪ Response detail:

▪ • Result envelope:

- data: response payload
 - isSuccess: boolean.
 - type: string or null.
 - errors: array of error items.
-
- totalRows: (int) response items number.
 - data array:
 - eventId: (int) event identifier.
 - eventDate: (datetime) event date.
 - plateNumber: (string) plate number.
 - globalConfidence: (decimal) entry location description.
 - cameraId: (int) camera identifier.
 - cameraName: (string) camera name.
 - computerId: (int) computer identifier.

- locationId: (int) location identifier.
- locationName: (string) location name.
- countryId: (int) country identifier.
- countryName: (string) country name.
- speed: (decimal) speed detected. If value is -1.0 speed is not detected.
- speedConfidence: (int) speed confidence. If speed value is -1.0 confidence will be -1
- direction: (int) lane direction. Values -1 (Going), 0 (Undetected) y 1 (Coming)
- cutImagePath: (string) image path.
- cutImage: (string) image blob.
- anprImagePath: (string) image path.
- anprImage: (string) image blob.
- redLightState: (int) red light state. 0 (Unknown), 1 (Green), 2 (Amber) y 3 (Red).
- redLightDescription: (string) red light description.
- blComments: (string) always return empty.
- comments: (string) event comment.
- laneId: (int) lane identifier.
- laneName: (string) lane name.
- analyticClassificationId: (int) classification identifier.
- analyticClassificationDescription: (string) classification name.
- analyticClassificationConfidence: (decimal) classification confidence.
- analyticMake: (string) make name.
- analyticMakeConfidence: (decimal) make confidence.
- analyticModel: (string) model name.
- analyticModelConfidence: (decimal) make confidence.
- analyticColor: (string) colour name.
- analyticColorConfidence: (decimal) make confidence.
- lastExportDate: (datetime) not used.

- characterHeight: (decimal) character height.
- engine: engine. LPR or ACCR.
- extraInfo: (string) only for ACCR.
- extrainfo2: (string) only for ACCR.
- validationUser: (string) not used.
- validationDate: (datetime) not used.
- gpsLatitude: (decimal) gps latitude.
- gpsLongitude: (decimal) gps longitude.
- personFirstname: (string) first name of the person linked to the vehicle.
- personSurname: (string) surname of the person linked to the vehicle.
- personSurname2: (string) second surname of the person linked to the vehicle.
- personPicture: (string) image of the person linked to the vehicle.
- directionDescription: (string) direction description. Undetected, Going or Coming.
- contrast: (decimal) contrast value.
- brightness: (decimal) brightness value.
- charConfidence (string) concatenate confidence values.
- processingTime: (int) processing time.
- nloEventName: (string) event name.
- dataSourceCamera: (string) camera source. 'L' Anpr camera or 'O' Orchestration camera.
- cutTop: (int) plate number coordinates.
- cutLeft: (int) plate number coordinates.
- cutRight: (int) plate number coordinates.
- cutBottom: (int) plate number coordinates.
- classificationTop: (int) image subject coordinates.
- classificationLeft: (int) image subject coordinates.
- classificationRight: (int) image subject coordinates.

- classificationBottom: (int) image subject coordinates.
- cutTopRelative: (decimal) image subject relative coordinates.
- cutLeftRelative: (decimal) image subject relative coordinates.
- cutRightRelative: (decimal) image subject relative coordinates.
- cutBottomRelative: (decimal) image subject relative coordinates.
- widthObjectEvent: (int) only for Orchestr[AI]tor. Object width.
- heightObjectEvent: (int) only for Orchestr[AI]tor. Object height.
- eventInsertionType: (string) event insertion type. “, ‘Automatic’ or ‘Manual’

▪ **Response example:**

```
{
  "data": {
    "items": [
      {
        "eventId": 174132,
        "eventDate": "2026-02-16T17:26:05.653",
        "plateNumber": "6621FYL",
        "globalConfidence": 97.564376831054688,
        "cameraId": 4,
        "cameraName": "Ajunta-Video-1",
        "computerId": 1,
        "locationId": 2,
        "locationName": "AjuntaCamVideo",
        "countryId": 101,
        "countryName": "Spain",
        "speed": -1.0,
        "speedConfidence": -1,
        "direction": 0,
        "cutImagePath": "c:\\tmp\\imgs\\RH415_cut.jpg",
        "cutImage": "data:image/jpeg;base64,/9j/4AAQSkZIRgA",
        "anprImagePath": "c:\\tmp\\imgs\\RH415.jpg",
        "anprImage": "data:image/jpeg;base64,/9j/4AAQSkZIRgA",
        "redLightState": 0,
        "redLightDescription": "Unknown",
        "blComments": "",
        "comments": null,
        "laneId": 1,
        "laneName": "LANE 1",
        "analyticClassificationId": 2,
        "analyticClassificationDescription": "CAR",

```

Neural Platform – NS Backend web API integration

```
"analyticClassificationConfidence": 95.7558364868164,
"analyticMake": "FIAT",
"analyticMakeConfidence": 78.102462768554688,
"analyticModel": "UNKNOWN",
"analyticModelConfidence": 0,
"analyticColor": "WHITE",
"analyticColorConfidence": 61.859466552734375,
"lastExportDate": null,
"characterHeight": 22.857143402099609,
"engine": "LPR",
"extraInfo": null,
"extraInfo2": null,
"validationUser": "Autom.",
"validationDate": "2026-02-16T17:26:05.653",
"gpsLatitude": 0,
"gpsLongitude": 0,
"personFirstname": null,
"personSurname": null,
"personSurname2": null,
"personPicture": null,
"directionDescription": "UNDETECTED",
"contrast": 98.823532104492188,
"brightness": 86.666664123535156,
"charConfidence": ";92.08;93.48;93.82;84.32;93.81;93.10;92.59",
"processingTime": 23,
"nloEventName": "",
"dataSourceCamera": "L",
"cutTop": 427,
"cutLeft": 910,
"cutRight": 1030,
"cutBottom": 451,
"classificationTop": 1,
"classificationLeft": 770,
"classificationRight": 1533,
"classificationBottom": 550,
"cutTopRelative": 0.39537037037037037,
"cutLeftRelative": 0.47395833333333331,
"cutRightRelative": 0.53645833333333337,
"cutBottomRelative": 0.41759259259259257,
"widthObjectEvent": 0,
"heightObjectEvent": 0,
"eventInsertionType": "Automatic"
}
],

"totalRows": 1
},

"isSuccess": true,
"type": null,
"errors": []
```

```
}
```

Response errors:

- 400 Bad Request (validation error).

Returned when request validation fails (for example, dateFrom >= dateTo).

Response example:

```
{  
  "data": null,  
  "isSuccess": false,  
  "type": "Validation",  
  "errors": [  
    {  
      "code": "ExportEvents.DateFromLessThanDateTo",  
      "detail": "DateFrom must be less than DateTo."  
    }  
  ]  
}
```

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid/expired token).

Response body may be empty.

- 403 Forbidden.

Returned when the token is valid but does not contain required permission ("ver_events").

Response body may be empty.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

6.2. Get event by identifier

The endpoint to get an event by its identifier return an event by its identifier. Returns the same object as one item in Get Events. The next table has a parameters detail list.

Parameters marked with “*” are mandatory in the request.

- **Verb:** GET
- **Path:** /nlapi/api/v2/Events/get-event-by-id
- **Permissions:** login user needs permission with name command “ver_events”.
- **Parameters:**

Param name	Type	Default values	Format or allowed values	Description
* eventId	int			Event identifier

* computerId	int			Computer identifier
--------------	-----	--	--	---------------------

■ Response detail:

The response returns the same object that one item in method [Get Events](#)

```
{
  "data": {
    "eventId": 174132,
    "eventDate": "2026-02-16T17:26:05.653",
    "plateNumber": "6621FYL",
    "globalConfidence": 97.564376831054688,
    "cameraId": 4,
    "cameraName": "Ajunta-Video-1",
    "computerId": 1,
    "locationId": 2,
    "locationName": "AjuntaCamVideo",
    "countryId": 101,
    "countryName": "Spain",
    "speed": -1.0,
    "speedConfidence": -1,
    "direction": 0,
    "cutImagePath": "c:\\tmp\\imgs\\RH415_cut.jpg",
    "cutImage": "data:image/jpeg;base64,/9j/4AAQSkZIRgA",
    "anprImagePath": "c:\\tmp\\imgs\\RH415.jpg",
    "anprImage": "data:image/jpeg;base64,/9j/4AAQSkZIRgA",
    "redLightState": 0,
    "redLightDescription": "Unknown",
    "blComments": "",
    "comments": null,
    "laneId": 1,
    "laneName": "LANE 1",
    "analyticClassificationId": 2,
    "analyticClassificationDescription": "CAR",
    "analyticClassificationConfidence": 95.7558364868164,
    "analyticMake": "FIAT",
    "analyticMakeConfidence": 78.102462768554688,
    "analyticModel": "UNKNOWN",
    "analyticModelConfidence": 0,
    "analyticColor": "WHITE",
    "analyticColorConfidence": 61.859466552734375,
    "lastExportDate": null,
    "characterHeight": 22.857143402099609,
    "engine": "LPR",
    "extraInfo": null,
    "extraInfo2": null,
    "validationUser": "Autom.",
    "validationDate": "2026-02-16T17:26:05.653",
    "gpsLatitude": 0,
    "gpsLongitude": 0,
  }
}
```

```

"personFirstname": null,
"personSurname": null,
"personSurname2": null,
"personPicture": null,
"directionDescription": "UNDETECTED",
"contrast": 98.823532104492188,
"brightness": 86.666664123535156,
"charConfidence": ";92.08;93.48;93.82;84.32;93.81;93.10;92.59",
"processingTime": 23,
"nloEventName": "",
"dataSourceCamera": "L",
"cutTop": 427,
"cutLeft": 910,
"cutRight": 1030,
"cutBottom": 451,
"classificationTop": 1,
"classificationLeft": 770,
"classificationRight": 1533,
"classificationBottom": 550,
"cutTopRelative": 0.39537037037037037,
"cutLeftRelative": 0.47395833333333331,
"cutRightRelative": 0.53645833333333337,
"cutBottomRelative": 0.41759259259259257,
"widthObjectEvent": 0,
"heightObjectEvent": 0,
"eventInsertionType": "Automatic"
},
"isSuccess": true,
"type": null,
"errors": []
}

```

Response errors:

- 400 Bad Request (validation error).

Returned when one or more input parameters are invalid (for example, eventId <= 0 or computerId <= 0).

Response example:

```

{
  "data": null,
  "isSuccess": false,
  "type": "Validation",
  "errors": [
    {

```

Neural Platform – NS Backend web API integration

```
"code": "GetEventById.EventIdInvalid",
"detail": "A valid event ID is required."
},
{
  "code": "GetEventById.ComputerIdInvalid",
  "detail": "A valid computer ID is required."
}
]
}
```

- 404 Not Found.

Returned when no event exists for the provided eventId and computerId.

Response example:

```
{
  "data": null,
  "isSuccess": false,
  "type": "NotFound",
  "errors": [
    {
      "code": "GetEventById.NotFound",
      "detail": "Event not found for the given event ID and computer ID."
    }
  ]
}
```

- 401 Unauthorized.

Returned when the user is not authenticated (missing/invalid token).

- 403 Forbidden.

Returned when the token is valid but does not contain any required permission for this endpoint.

- 500 Internal Server Error.

Returned on unexpected server errors.

Response example:

```
{
  "error": {
    "message": "An unexpected error occurred while processing the request.",
    "data": "",
    "exception": "Exception"
  }
}
```

NOTES:

If an external system needs more images or videos for an event/incidence, system can do another call to API using [Images and video endpoints](#)